

Comandos de cadenas

Crear una cadena

```
cadena = "Hola, soy una cadena en Python"
```

Longitud de una cadena (len)

```
cadena = "Hola, mundo"
longitud = len(cadena)
print(longitud) # Salida: 11
```

Acceder a caracteres por índice

```
cadena = "Python"
primer_caracter = cadena[0] # P
segundo_caracter = cadena[1] # y
ultimo_caracter = cadena[-1] # n
```

Rebanar una cadena (slicing)

```
cadena = "Python"
subcadena = cadena[1:4] # yth
```

Concatenar cadenas

```
nombre = "Juan"
apellido = "Pérez"
nombre_completo = nombre + " " + apellido
print(nombre_completo) # Salida: Juan Pérez
```

Métodos de cadenas

```
cadena = "Hola, mundo"
print(cadena.upper()) # Salida: HOLA, MUNDO
print(cadena.lower()) # Salida: hola, mundo
print(cadena.capitalize()) # Salida: Hola, mundo
print(cadena.replace("Hola", "Adiós"))
# Salida: Adiós, mundo
```

Buscar subcadenas (find, index)

```
cadena = "Hola, mundo"
posicion1 = cadena.find("mundo") # 6
posicion2 = cadena.index("mundo") # 6
```

Dividir una cadena (split)

```
cadena = "Hola,amigo,cómo,estás"
lista_palabras = cadena.split(",")
# ['Hola', 'amigo', 'cómo', 'estás']
```

Eliminar espacios en blanco (strip)

```
cadena = "  Hola, mundo  "
cadena_limpia = cadena.strip() # "Hola, mundo"
```

Comprobar si una cadena comienza o termina con cierto texto (startswith, endswith)

```
cadena = "Hola, mundo"
comienza_con_hola = cadena.startswith("Hola") # True
termina_con_mundo = cadena.endswith("mundo") # True
```

Comandos de diccionarios

Crear un diccionario

```
mi_diccionario = {
    "nombre": "Juan", "edad": 30,
    "profesion": "Ingeniero"}
```

Acceder a valores por clave

```
mi_diccionario = {"nombre": "Juan", "edad": 30,
    "profesion": "Ingeniero"}
nombre = mi_diccionario["nombre"]
edad = mi_diccionario["edad"]
print(nombre) # Salida: "Juan"
print(edad) # Salida: 30
```

Modificar valores en diccionario

```
mi_diccionario = {"nombre": "Juan", "edad": 30,
    "profesion": "Ingeniero"}
mi_diccionario["edad"] = 31
print(mi_diccionario) # Salida: {"nombre": "Juan",
    "edad": 31, "profesion": "Ingeniero"}
```

Verificar si una clave existe en el diccionario

```
mi_diccionario = {"nombre": "Juan", "edad": 30,
    "profesion": "Ingeniero"}
existe_nombre = "nombre" in mi_diccionario
existe_apellido = "apellido" in mi_diccionario
print(existe_nombre) # Salida: True
print(existe_apellido) # Salida: False
```

Comandos de tuplas

Crear una tupla

```
mi_tupla = (1, 2, 3, 4, 5)
```

Acceder a elementos de una tupla por índice

```
mi_tupla = (10, 20, 30, 40, 50)
primer_elemento = mi_tupla[0] # 10
ultimo_elemento = mi_tupla[-1] # 5
```

Rebanar una tupla (slicing)

```
mi_tupla = (10, 20, 30, 40, 50)
subtupla = mi_tupla[1:4] # (20, 30, 40)
```

Longitud de una tupla (len)

```
mi_tupla = (1, 2, 3, 4, 5)
longitud = len(mi_tupla)
print(longitud) # Salida: 5
```

Comandos de listas

Acceder a elementos por índice

```
numeros = [1, 2, 3, 4, 5]
primer_elemento = numeros[0] # 1
segundo_elemento = numeros[1] # 2
ultimo_elemento = numeros[-1] # 5
```

Rebanar una lista (slicing)

```
numeros = [1, 2, 3, 4, 5]
sublista = numeros[1:4] # [2, 3, 4]
```

Modificar elementos de una lista

```
numeros = [1, 2, 3, 4, 5]
numeros[0] = 10
print(numeros) # Salida: [10, 2, 3, 4, 5]
```

Agregar elementos al final de una lista (append)

```
numeros = [1, 2, 3, 4, 5]
numeros.append(6)
print(numeros) # Salida: [1, 2, 3, 4, 5, 6]
```

Insertar elementos en una posición específica (insert)

```
numeros = [1, 2, 3, 4, 5]
numeros.insert(2, 10)
print(numeros) # Salida: [1, 2, 10, 3, 4, 5]
```

Eliminar elementos por valor (remove)

```
numeros = [1, 2, 3, 4, 5]
numeros.remove(3)
print(numeros) # Salida: [1, 2, 4, 5]
```

Eliminar elementos por índice (del)

```
numeros = [1, 2, 3, 4, 5]
del numeros[0]
print(numeros) # Salida: [2, 3, 4, 5]
```

Extender una lista con otra (extend)

```
numeros = [1, 2, 3]
otros_numeros = [4, 5, 6]
numeros.extend(otros_numeros)
print(numeros) # Salida: [1, 2, 3, 4, 5, 6]
```

Contar elementos en una lista (count)

```
numeros = [1, 2, 2, 3, 2, 4, 5, 2]
cantidad_de_dos = numeros.count(2) # 4
```

Ordenar una lista (sort):

```
numeros = [5, 3, 1, 4, 2]
numeros.sort()
print(numeros) # Salida: [1, 2, 3, 4, 5]
```

Poner la lista al revés

```
numeros = [5, 3, 1, 4, 2]
numeros.reverse()
print(numeros) # Salida [2, 4, 1, 3, 5]
```

Lista con elementos

```
numeros = [1, 2, 3, 4, 5]
nombres = ["Ana", "Carlos", "María"]
mezcla = [10, "hola", True, 3.14]
```

Obtener componentes individuales

Importar datetime, date y time del módulo datetime

```
from datetime import datetime, date, time
```

Obtener la fecha y hora actual

```
fecha_hora_actual = datetime.now()
```

Obtener el año, mes, día, hora, minutos y segundos de la fecha y hora actual

```
anio = fecha_hora_actual.year
mes = fecha_hora_actual.month
dia = fecha_hora_actual.day
hora = fecha_hora_actual.hour
minutos = fecha_hora_actual.minute
segundos = fecha_hora_actual.second
```

```
print("Fecha:", dia, mes, anio)
print("Hora:", hora, minutos, segundos)
```

Operaciones con fechas

Importar date y timedelta del módulo datetime

```
from datetime import date, timedelta
```

Crear un objeto date con la fecha 2023-08-01 (año-mes-día)

```
fecha = date(2023, 8, 1)
```

Crear un timedelta de 3 días

```
duracion = timedelta(days=3)
```

Sumar el timedelta a la fecha para obtener una nueva fecha

```
nueva_fecha = fecha + duracion
print("Nueva Fecha:", nueva_fecha) # Resultado: 2023-08-04
```

Restar el timedelta a la fecha para obtener una nueva fecha anterior

```
fecha_anterior = fecha - duracion
print("Fecha Anterior:", fecha_anterior) # Resultado: 2023-07-29
```

Conversión de cadenas

Importar datetime, date y time del módulo datetime

```
from datetime import datetime, date, time
```

Cadena con la fecha y hora en formato "AAAA-MM-DD H:M:S"

```
cadena_fecha_hora = "2023-08-01 15:30:00"
```

Convertir la cadena a un objeto datetime

```
fecha_hora = datetime.strptime(cadena_fecha_hora, "%Y-%m-%d %H:%M:%S")
print(fecha_hora) # Resultado: 2023-08-01 15:30:00
```

Obtener días de la semana

Importar date del módulo datetime

```
from datetime import date
```

Obtener el día de la semana (0: lunes, 1: martes, ..., 6: domingo)

```
dia_semana = fecha.weekday()
print("Día de la semana:", dia_semana) # Resultado: 1 (martes)
```

Diferencia entre fechas

Importar date del módulo datetime

```
from datetime import date
```

Crear dos objetos date con las fechas 2023-08-01 y 2023-07-15

```
fecha1 = date(2023, 8, 1)
fecha2 = date(2023, 7, 15)
```

Calcular la diferencia entre las dos fechas

```
diferencia = fecha1 - fecha2
print("Diferencia en días:", diferencia.days) # Resultado: 17
```

Comandos NumPy

Reshape de arrays

Importar la librería NumPy

```
import numpy as np
```

Crear un array unidimensional del 1 al 12

```
array = np.arange(1, 13)
print(array)
# Resultado: [ 1  2  3  4  5  6  7  8  9 10 11 12]
```

Reshape del array en una matriz de 3x4

```
matriz = array.reshape(3, 4)
print(matriz)
# Resultado:
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
```

Operaciones estadísticas

Importar librería NumPy

```
import numpy as np
```

Crear un array de números aleatorios

```
array = np.random.randint(1, 101, size=5)
print(array) # Resultado: [65 10 73 49  9]
```

Obtener el valor máximo y mínimo del array

```
maximo = np.max(array)
minimo = np.min(array)
print(maximo, minimo) # Resultado: 73 9
```

Calcular la media y desviación estándar del array

```
media = np.mean(array)
desviacion_estandar = np.std(array)
print(media, desviacion_estandar)
# Resultado: 41.2 26.088470577796655
```

Creación de arrays con valores específicos

Importar la librería NumPy

```
import numpy as np
```

Crear un array de 0s de tamaño 4x3

```
array_ceros = np.zeros((4, 3))
print(array_ceros)
# Resultado:
# [[0. 0. 0.]
#  [0. 0. 0.]
#  [0. 0. 0.]
#  [0. 0. 0.]]
```

Crear un array de 1s de tamaño 2x4

```
array_unos = np.ones((2, 4))
print(array_unos)
# Resultado:
# [[1. 1. 1. 1.]
#  [1. 1. 1. 1.]]
```

Crear un array con valores espaciados uniformemente

```
array_espaciado = np.linspace(0, 1, 5)
print(array_espaciado) # Resultado: [0.    0.25 0.5   0.75 1.   ]
```

Concatenación de arrays

Importar librería NumPy

```
import numpy as np
```

Crear dos arrays

```
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])
```

Concatenar los arrays verticalmente (uno encima del otro)

```
concatenacion_vertical = np.vstack((array1, array2))
print(concatenacion_vertical)
# Resultado:
# [[1 2 3]
#  [4 5 6]]
```

Concatenar los arrays horizontalmente (uno al lado del otro)

```
concatenacion_horizontal = np.hstack((array1, array2))
print(concatenacion_horizontal) # Resultado: [1 2 3 4 5 6]
```

Copiar arrays

Importar librería NumPy

```
import numpy as np
```

Crear un array

```
array_original = np.array([1, 2, 3])
```

Crear una copia independiente del array

```
array_copia = array_original.copy()
```

Modificar el array copia

```
array_copia[0] = 10
```

El array original no se ve afectado por la modificación en la copia

```
print(array_original) # Resultado: [1 2 3]
print(array_copia)    # Resultado: [10  2  3]
```

Funciones de lógica booleana

Importar librería NumPy

```
import numpy as np
```

Crear un array

```
array = np.array([1, 2, 3, 4, 5])
```

Obtener un array de valores booleanos que indican si cada elemento es mayor que 3

```
es_mayor_que_3 = array > 3
print(es_mayor_que_3) # Resultado: [False False False  True  True]
```

Obtener un array solo con los elementos que cumplen una condición

```
elementos_mayor_que_3 = array[array > 3]
print(elementos_mayor_que_3) # Resultado: [4 5]
```

Ordenamiento de arrays

Importar librería NumPy

```
import numpy as np
```

Crear un array desordenado

```
array = np.array([3, 1, 5, 2, 4])
```

Ordenar el array en manera ascendente

```
array_ordenado = np.sort(array)
print(array_ordenado) # Resultado: [1 2 3 4 5]
```

Obtener los índices que ordena el array

```
indices_ordenados = np.argsort(array)
print(indices_ordenados) # Resultado: [1 3 0 4 2]
```